

Bước đầu tìm hiểu thuật toán phi tối ưu

Yang Pei, Trường Trung học số 4 Bắc Kinh

2000

Nguồn gốc: A preliminary study of non-optimal algorithms

IOI China candidate team papers / 2000 / Yang Pei.pdf

Tóm tắt

Bài viết giới thiệu lý thuyết cơ bản của các thuật toán phi tối ưu, tổng kết điều kiện áp dụng và một số kỹ thuật sử dụng thuật toán tham lam. Trên cơ sở đó, bài viết trình bày thuật toán tìm kiếm tabu và các ví dụ ứng dụng, chỉ ra một số phạm vi áp dụng của phương pháp ngẫu nhiên hóa, và tổng kết ưu thế của các thuật toán phi tối ưu.

Từ khóa. Tham lam; ngẫu nhiên hóa; tối ưu hóa; tìm kiếm cục bộ.

1 Khái luận

Các bài toán tin học hiện đại có thể chia thành hai loại: một loại là các bài toán lớp P , tức đã có thuật toán hiệu quả; loại còn lại là các bài toán NPC, tức hiện nay chưa tìm được thuật toán hiệu quả. Để xử lý loại thứ hai, ta đưa vào khái niệm thuật toán phi tối ưu. Phần này trình bày khái niệm NPC, cũng như sự cần thiết của việc nêu ra các thuật toán phi tối ưu.

1.1 Mở đầu

Tin học là một ngành khoa học rất mới, ra đời vào nửa sau thế kỷ 20. Sự ra đời của toán học, vật lý, hóa học và các ngành tương tự được xây dựng trên cơ sở nghiên cứu thuần túy theo hứng thú của một số nhà khoa học; ở giai đoạn đầu, công việc của họ chủ yếu mang tính lý thuyết. Điểm khác biệt lớn nhất của tin học là ngay từ khi ra đời, nó đã gắn bó chặt chẽ với ứng dụng thực tế. Mỗi thuật toán được đề xuất đều có bối cảnh ứng dụng rộng rãi. Từ khi Olympic Tin học Quốc tế (IOI) được tổ chức lần đầu năm 1989, cuộc thi đã trải qua 11 kỳ, và phong cách đề bài trong thời gian đó cũng phát triển theo một quá trình không thẳng. Nhưng nhìn tổng thể, xu thế chung là tiến gần hơn tới thực tế: mỗi bài đều có một bối cảnh thực tế và kiểm tra khả năng của thí sinh trong việc trừu tượng hóa mô hình toán học từ vấn đề thực.

Tuy nhiên, trong rất nhiều vấn đề thực tế, những bài toán P thật sự có thuật toán hiệu quả chỉ chiếm thiểu số, còn phần lớn những bài toán gây khó khăn cho con người lại là các bài toán NPC. Khi chưa có thuật toán hiệu quả, muốn giải các bài toán NPC ta chỉ có thể dùng một số thuật toán phi tối ưu để tìm nghiệm xấp xỉ trong độ phức tạp thời gian chấp nhận được. Vì vậy, để kiểm tra năng lực của thí sinh trong việc giải loại bài này, trong các cuộc thi những năm gần đây đã xuất hiện ngày càng nhiều bài có cách chấm điểm không thông thường, như chấm theo từng mức. Ngoài ra, trong quá trình thi, với một số

bài tạm thời chưa nghĩ ra thuật toán hiệu quả, hoặc việc cài đặt thuật toán hiệu quả tương đối khó, dùng thuật toán phi tối ưu cũng có thể đạt kết quả khá tốt (xem bảng 1). Từ hai lý do trên có thể thấy rằng thuật toán phi tối ưu vẫn còn nhiều đất dụng võ trong tương lai; việc nghiên cứu và tổng kết chúng là cần thiết.

Năm	Cuộc thi	Bài toán	Thuật toán tốt	Loại bài toán
1997	IOI	Xe thám hiểm Sao Hỏa	Tham lam	Luồng cực đại
		Gắn nhãn bản đồ	Tham lam/ngẫu nhiên hóa	NPC
		Container	Xác suất + tham lam	Quy hoạch ngẫu nhiên
		Rết	Dựng thuật toán xấp xỉ	
		Trò chơi HEX	Dựng thuật toán xấp xỉ	Trò chơi
1998	NOI	Tính toán song song	Tham lam + ngẫu nhiên hóa	Tìm kiếm lớn
1999	Trại đông	Cải tạo mê cung	Tham lam + ngẫu nhiên hóa	Quy hoạch động
	NOI	Xâu 01	Ngẫu nhiên hóa	Đường đi dài nhất
	Bài tập đội tuyển	Bảo vệ Trái Đất	Ngẫu nhiên hóa	NPC
	IOI	Thành phố ngầm	Tham lam	
		Chia đều quân bài	Tham lam/ngẫu nhiên hóa	

Table 1: Một số bài toán mà thuật toán phi tối ưu cho hiệu quả tốt

1.2 Khái niệm cơ bản

1.2.1 Quan hệ giữa bài toán khả thi và bài toán tối ưu

Các bài toán áp dụng thuật toán phi tối ưu có thể chia đơn giản thành hai loại: bài toán khả thi và bài toán tối ưu. Tuy khi chọn thuật toán cụ thể ta cần phân biệt và xử lý hai loại này riêng, về bản chất chúng thống nhất với nhau và đều có thể quy về bài toán quyết định sau.

Định nghĩa 1.2.1. Nếu mỗi thể hiện của một bài toán chỉ có hai câu trả lời “có” hoặc “không”, thì bài toán đó được gọi là **bài toán quyết định**.

Bài toán khả thi có thể trực tiếp chuyển thành bài toán quyết định “có tồn tại nghiệm hay không”. Với bài toán tối ưu, ta có thể chuyển thành một số bài toán quyết định dạng “có tồn tại nghiệm tốt hơn nghiệm hiện tại hay không”. Theo nghĩa này, bài toán khả thi là một trường hợp con của bài toán tối ưu, và ta chỉ cần tập trung nghiên cứu loại sau.

1.2.2 Khái niệm lân cận

Định nghĩa 1.2.2. Với một bài toán tối ưu, xét tập D gồm tất cả nghiệm khả thi. Một ánh xạ

$$N : S \in D \mapsto N(S) \in 2^D$$

được gọi là **ánh xạ lân cận**, trong đó 2^D biểu thị tập tất cả các tập con của D . Tập $N(S)$ được gọi là **lân cận** của S , và $S' \in N(S)$ được gọi là một **hàng xóm** của S .

Trên thực tế, những thuật toán đơn giản truyền thống như leo đồi và tham lam đều được xây dựng trên cơ sở tìm kiếm trong lân cận. Rõ ràng, các phương pháp trên chỉ có thể thu được nghiệm tối ưu cục bộ, không bảo đảm thu được tối ưu toàn cục. Một trong những mục tiêu của thuật toán phi tối ưu là dùng chi phí nhỏ để nhảy ra khỏi điểm tối ưu cục bộ, từ đó tiếp cận tối ưu toàn cục nhiều nhất có thể.

1.3 Phân loại thuật toán phi tối ưu

Thuật toán phi tối ưu có thể chia đơn giản thành hai loại: thuật toán một bước và thuật toán cải tiến.

1.3.1 Thuật toán một bước

Đặc điểm của loại thuật toán này là: nó không lựa chọn giữa hai nghiệm khả thi; trong các vòng lặp chưa kết thúc, trạng thái hiện tại lại có thể chưa phải là nghiệm khả thi; khi thuật toán kết thúc mới thu được một nghiệm khả thi. Độ phức tạp thời gian của thuật toán như vậy thường dễ chấp nhận.

Ví dụ điển hình là thuật toán tham lam cho bài xe thám hiểm Sao Hỏa: mỗi xe chọn một tuyến có thể chở nhiều quặng nhất, cho tới khi phân phối xong tuyến đường cho tất cả xe. Thuật toán không so sánh lựa chọn giữa hai nghiệm khả thi, và khi kết thúc thu được một nghiệm khả thi. Cần chú ý rằng khi giải bài toán khả thi, trong quá trình chạy thuật toán có thể phát hiện không thể đạt được nghiệm khả thi cuối cùng; khi đó cần quay lui đơn giản, hoặc phá bỏ làm lại từ đầu nếu dùng phương pháp ngẫu nhiên hóa. Ví dụ là bài “Xâu 01” của NOI 1999.

1.3.2 Thuật toán cải tiến

Trong thuật toán cải tiến, quá trình lặp đi từ một nghiệm khả thi sang một nghiệm khả thi khác. Thông thường thuật toán so sánh hai nghiệm để chọn nghiệm tốt hơn, rồi dùng nó làm điểm xuất phát mới cho lần lặp tiếp theo, cho tới khi thỏa một yêu cầu nhất định. Loại thuật toán này thường áp dụng cho bài toán tối ưu. Chẳng hạn, tìm kiếm cục bộ hoặc leo đồi đều là một dạng thuật toán cải tiến. Ngoài ra, khi sử dụng ngẫu nhiên hóa, ví dụ trong bài cải tạo mê cung, ta cần lặp lại việc sinh ngẫu nhiên nghiệm khả thi rồi cuối cùng chọn nghiệm tốt nhất trong số đó; cách làm này cũng có thể xem là một thuật toán cải tiến.

1.4 Phân tích hiệu năng của thuật toán phi tối ưu

Thuật toán phi tối ưu có nhiều ưu điểm, nhưng khuyết điểm lớn nhất là không bảo đảm thu được nghiệm tối ưu toàn cục. Vì vậy việc đánh giá thuật toán trở nên rất quan trọng. Để đánh giá một thuật toán phi tối ưu cần hai tiêu chuẩn: thứ nhất là xem nghiệm của nó gần nghiệm tối ưu đến mức nào, đây là điều kiện cần bản. Nếu nghiệm thu được cách nghiệm tối ưu quá xa, với các cách chấm điểm hiện có thì điểm số sẽ khá thấp, chưa nói đến việc áp dụng cho những bài yêu cầu nghiệm tối ưu. Thứ hai là xét tính ổn định của thuật toán, điều này cũng rất cần thiết. Sau đây là vài phương pháp đơn giản để kiểm thử thuật toán.

1.4.1 Phân tích trường hợp xấu nhất

Với bất kỳ thuật toán nào, ta đều phải xét độ phức tạp thời gian và không gian trong trường hợp xấu nhất. Nhưng với thuật toán phi tối ưu, ta còn cần xét hiệu quả nghiệm của nó trong trường hợp xấu nhất. Khi dùng phân tích trường hợp xấu nhất để đánh giá hiệu quả thuật toán, chỉ tiêu là khoảng cách giữa giá trị nghiệm tính được và giá trị nghiệm tối ưu; khoảng cách càng nhỏ thì thuật toán càng tốt.

Tuy nhiên, phân tích trường hợp xấu nhất không thể đánh giá toàn diện chất lượng thuật toán. Một số thuật toán có hiệu quả rất kém ở quy mô nhỏ, nhưng không thể nói chắc rằng thuật toán đó nhất định kém; ví dụ thuật toán của Qian Wenjie trong bài container. Ngoài ra, tìm được trường hợp xấu nhất của một thuật toán không dễ, cần nhiều kỹ thuật toán học, và điều này cũng hạn chế việc áp dụng phương pháp phân tích này.

1.4.2 Phân tích xác suất

Phương pháp phân tích xác suất thống kê là phương pháp lý thuyết: nó giả thiết dữ liệu của thể hiện tuân theo một phân bố xác suất nhất định. Dưới giả thiết về phân bố dữ liệu này, ta nghiên cứu hiệu quả trung bình của thuật toán hoặc của nghiệm. Phương pháp phân tích này rất quan trọng trong việc ước lượng tính khả thi của thuật toán ngẫu nhiên hóa. Chẳng hạn, với thuật toán kiểm tra số nguyên tố ngẫu nhiên

hóa, phân tích xác suất cung cấp cơ sở lý thuyết cho tính ổn định của thuật toán. Nhưng phân tích xác suất là một phương pháp lý thuyết; nó đòi hỏi hiểu sâu bản thân bài toán, nắm được cách xây dựng mô hình xác suất và lý thuyết xác suất, tức cần nền tảng toán học khá mạnh. Với một số bài toán phức tạp, không thể hoàn thành phân tích trong phòng thi, nên phương pháp này có hạn chế lớn.

1.4.3 Phân tích bằng tính toán quy mô lớn

Phân tích bằng tính toán quy mô lớn chính là cách thường gọi là dùng dữ liệu kiểm thử để kiểm thử thuật toán. Nó có thể đánh giá nhiều thuật toán và so sánh hiệu quả của chúng. Dựa trên kết quả tính toán của từng thuật toán, ta có thể dùng phương pháp đơn giản hoặc phương pháp thống kê để so sánh hiệu năng, mà không cần biết trước nghiệm tối ưu của từng thể hiện. Khi dùng phương pháp này để phân tích thuật toán, cần sinh một số dữ liệu kiểm thử có tính đại diện. Về cách làm cho dữ liệu kiểm thử toàn diện và chính xác, có thể tham khảo bài của Yang Fan, “Tính chính xác, tính toàn diện, tính thẩm mỹ: ba yếu tố trong thiết kế dữ liệu kiểm thử”.

2 Bước đầu tìm hiểu thuật toán phi tối ưu

2.1 Thuật toán tham lam

2.1.1 Khái niệm

Thuật toán tham lam xuất phát từ một nghiệm ban đầu nào đó của bài toán và từng bước tiến gần tới mục tiêu cho trước, nhằm tìm được nghiệm tốt hơn nhanh nhất có thể. Khi tới một bước mà thuật toán không thể tiếp tục tiến lên, thuật toán dừng. Lúc này ta thu được một nghiệm của bài toán, nhưng không bảo đảm nghiệm cuối cùng là tối ưu. Trong thuật toán cải tiến, thuật toán tham lam phát triển thành phương pháp leo đồi.

2.1.2 Đặc điểm và phạm vi sử dụng

Ưu điểm của thuật toán tham lam là độ phức tạp thời gian cực thấp. Điểm khác giữa thuật toán tham lam và các thuật toán tối ưu hóa khác là: nó không thể rút lui, có thể có hậu hiệu, và trong trường hợp chung không thỏa nguyên lý tối ưu. Những đặc điểm này quyết định phạm vi áp dụng của nó: thông thường nó không thích hợp để giải bài toán khả thi, mà chỉ thích hợp với bài toán tối ưu khi tương đối dễ thu được nghiệm khả thi. Ở đây, “tương đối dễ thu được nghiệm khả thi” nghĩa là sau khi chọn chiến lược hiện tại, các bước phía sau sẽ không, hoặc rất ít khi, rơi vào tình trạng vô nghiệm.

Ngoài ra, với các bài tương tác xuất hiện trong những năm gần đây, thuật toán tham lam là một lựa chọn khá tốt. Lý do là trong loại bài này, kết quả của một chiến lược được đưa ra dần trong quá trình làm bài; ta không thể biết trước kết quả của chiến lược đã chọn. Điều này rất phù hợp với đặc điểm của thuật toán tham lam: không xét trước kết quả chiến lược và có hậu hiệu. Dĩ nhiên, thuật toán tham lam còn có thể cung cấp cận ban đầu tốt cho thuật toán tìm kiếm.

2.1.3 Kỹ thuật sử dụng thuật toán tham lam

Kết hợp tham lam với thuật toán tối ưu. Mặc dù thuật toán tham lam có ưu thế nhất định, trong trường hợp chung nó vẫn không thu được nghiệm tối ưu. Vì vậy, để giảm tác dụng phụ do tham lam gây ra và làm nghiệm cuối cùng gần nghiệm tối ưu hơn, ta nên dùng các thuật toán tối ưu hiệu quả, như quy hoạch động, ở nhiều vị trí nhất có thể trong thuật toán.

Chẳng hạn, trong bài xe thám hiểm Sao Hỏa, khi ta không tìm được thuật toán hiệu quả để các xe thám hiểm hỗ trợ nhau và phân phối nhiệm vụ tổng hợp, ta chỉ có thể dùng thuật toán tham lam: để mỗi xe tự vận hành và tìm một tuyến đường giúp chính nó thu được nhiều quặng nhất. Trong quá trình đó, ta có thể dùng quy hoạch động để bảo đảm mỗi xe đi theo tuyến hiện tại tốt nhất. Tuy tổng các nghiệm tối ưu cục bộ không bằng nghiệm tối ưu toàn cục, cách làm này vẫn bù đắp khá nhiều nhược điểm của việc dùng tham lam ở mức toàn cục; xem chi tiết trong tài liệu [1], Vol.1 No.1.

Trong bài “Bảo vệ Trái Đất” của Shao Zheng, với bài toán con dùng số lượng tên lửa hữu hạn để hạ thấp chiến lực của hạm đội đến mức thấp nhất, lời giải dùng thuật toán tham lam. Nhưng khi phân phối số tên lửa tấn công từng hạm đội, lời giải dùng quy hoạch động. Như vậy, ở cục bộ dùng tham lam, còn ở tổng thể dùng thuật toán “tối ưu”, và hiệu quả thu được cũng khá tốt; xem báo cáo lời giải của Shao Zheng trong tài liệu [7].

Lựa chọn trọng số trong thuật toán tham lam. Cốt lõi của thuật toán tham lam là trong các chiến lược có thể chọn, chọn chiến lược có trọng số tốt nhất làm chiến lược hiện tại. Vì vậy chất lượng của thuật toán tham lam chủ yếu do cách xác định trọng số quyết định.

Thuật toán xác định trọng số nên tuân theo hai nguyên tắc:

1. Cố gắng chứa thông tin về ảnh hưởng của việc chọn chiến lược này đối với toàn bộ bài toán. Rõ ràng, thông tin chứa càng nhiều thì trọng số càng gần với giá trị thật của chiến lược, và kết quả cuối cùng càng gần nghiệm tối ưu. Nếu trọng số chứa toàn bộ ảnh hưởng của việc chọn chiến lược này đối với bài toán, thuật toán sẽ thoái hóa thành thuật toán tìm kiếm.
2. Độ phức tạp tính trọng số không được quá cao, nếu không thuật toán sẽ mất ưu điểm duy nhất của tham lam.

Có thể thấy hai tiêu chuẩn trên mâu thuẫn với nhau. Mục tiêu của ta là tìm điểm cân bằng trong cặp mâu thuẫn đó.

2.1.4 Cải tiến thuật toán tham lam

Nhược điểm của thuật toán tham lam là hiệu quả nghiệm tương đối kém, còn ưu thế lớn nhất là độ phức tạp thời gian cực thấp, thường thấp hơn rất nhiều so với giới hạn của đề bài. Vậy tại sao không dùng thêm một phần thời gian để cải thiện nghiệm mục tiêu? Đây chính là sự cần thiết của việc cải tiến thuật toán tham lam. Ở đây ta thảo luận một thuật toán kết hợp tham lam với chiến lược tìm kiếm.

Thuật toán tìm kiếm cục bộ. Trước hết xét lại thuật toán leo đồi, tức chiến lược tìm kiếm cục bộ thuần túy.

Thuật toán 2.4.1: tìm kiếm cục bộ.

STEP1. Chọn một nghiệm khả thi ban đầu x_0 . Ghi nghiệm tốt nhất hiện tại $x_{\text{best}} := x_0$, đặt $P := N(x_{\text{best}})$.

STEP2. Nếu $P = \emptyset$ hoặc thỏa một tiêu chuẩn dừng khác, xuất kết quả và dừng. Ngược lại, chọn một tập S trong $N(x_{\text{best}})$ và lấy nghiệm tốt nhất x_{now} trong S . Nếu $f(x_{\text{now}}) < f(x_{\text{best}})$, đặt $x_{\text{best}} := x_{\text{now}}$ và $P := N(x_{\text{best}})$; nếu không, đặt $P := P - S$. Lặp lại STEP2.

Trong tìm kiếm cục bộ, nghiệm khả thi ban đầu ở STEP1 có thể được tìm bằng thuật toán tham lam hoặc thuật toán khác. Hiệu quả của thuật toán phụ thuộc vào kích thước của S .

Một ví dụ về tìm kiếm tabu. Sau đây ta đưa vào một thuật toán tìm kiếm tabu để cải tiến tìm kiếm cục bộ. Một tư tưởng quan trọng của nó là đánh dấu các nghiệm tối ưu cục bộ đã thu được và tránh các nghiệm tối ưu cục bộ đó trong những lần lặp tiếp theo. Trước hết xét một ví dụ.

Ví dụ 2.4: Bài “Xâu 01” của NOI 1999. Trước hết định nghĩa một hàm mục tiêu $f(x)$. Gọi $g_0(x, i)$ là số chữ số 0 trong L_0 ký tự liên tiếp bắt đầu từ vị trí i của nghiệm x . Gọi $g_1(x, i)$ là số chữ số 1 trong L_1 ký tự liên tiếp bắt đầu từ vị trí i của nghiệm x . Đặt

$$\phi_0(t) = \begin{cases} t - B_0, & t > B_0, \\ 0, & A_0 \leq t \leq B_0, \\ A_0 - t, & t < A_0, \end{cases} \quad \phi_1(t) = \begin{cases} t - B_1, & t > B_1, \\ 0, & A_1 \leq t \leq B_1, \\ A_1 - t, & t < A_1. \end{cases}$$

Khi đó

$$f(x) = \sum_{i=1}^{N-L_0+1} \phi_0(g_0(x, i)) + \sum_{i=1}^{N-L_1+1} \phi_1(g_1(x, i)).$$

Rõ ràng nhiệm vụ của ta là tìm một nghiệm x' sao cho $f(x') = 0$. Trong quá trình giải, mục tiêu là tìm nghiệm x làm $f(x)$ nhỏ nhất có thể.

Ta minh họa bằng một thể hiện:

$$N = 10, \quad A_0 = 1, \quad B_0 = 2, \quad L_0 = 3, \quad A_1 = 1, \quad B_1 = 1, \quad L_1 = 3.$$

Giả sử nghiệm ban đầu là $x_0 = (1111111111)$. Ánh xạ lân cận là phép lật một vị trí nào đó, và giá trị mục tiêu ban đầu là $f(x_0) = 24$. Trong các bảng sau, “giá trị” là giá trị mục tiêu sau khi thực hiện phép lật; dấu * đánh dấu thao tác được chọn, còn T đánh dấu thao tác đang bị tabu.

Bước 1, thao tác	1	2	3	4	5	6	7	8	9	10
Giá trị	22	20	18*	18	18	18	18	18	20	22
Độ dài tabu	0	0	0	0	0	0	0	0	0	0

Từ tập ứng viên ta chọn một thao tác tốt nhất: lật vị trí 3. Giá trị mục tiêu giảm từ 24 xuống 18, tức có cải thiện.

Bước 2, $x_1 = (1101111111), f(x_1) = 18$	1	2	3	4	5	6	7	8	9	10
Giá trị	17	16	24T	14	13	12*	12	12	14	16
Độ dài tabu	0	0	3	0	0	0	0	0	0	0

Vì ở bước 1 đã chọn lật vị trí 3, ta hy vọng phép đổi này sẽ không xuất hiện lại trong vài lần lặp sau, để tránh chu trình trong tính toán. Thao tác 3 được gọi là đối tượng tabu và bị cấm lật lại vị trí 3 trong 3 lần lặp; vị trí tương ứng ghi 3. Trong $N(x_1)$ lại xuất hiện thao tác 3 bị tabu, nên dùng T để đánh dấu và không chọn phép đổi này.

Bước 3, $x_2 = (1101101111), f(x_2) = 12$	1	2	3	4	5	6	7	8	9	10
Giá trị	11	10	18T	9	9	18T	8	7*	8	10
Độ dài tabu	0	0	2	0	0	3	0	0	0	0

Sau khi thao tác 6 mới chọn bị cấm, thao tác 3 đã bị cấm một lần và còn hai lần tabu.

Thao tác 3 được giải cấm.

Ở đây giá trị đánh giá của tất cả thao tác đều kém hơn giá trị ban đầu. Với tìm kiếm cục bộ thông thường, lúc này thuật toán đã đạt nghiệm tối ưu cục bộ và dừng. Nhưng hiện tại ta cho phép chọn một thao tác tốt nhất trong tập ứng viên: lật vị trí 10. Như vậy thuật toán có thể nhảy ra khỏi nghiệm tối ưu cục bộ hiện tại.

Bước 4, $x_3 = (1101101011), f(x_3) = 7$	1	2	3	4	5	6	7	8	9	10
Giá trị	6	5	13T	4*	4	12T	7	12T	5	6
Độ dài tabu	0	0	1	0	0	2	0	3	0	0

Bước 5, $x_4 = (1100101011), f(x_4) = 4$	1	2	3	4	5	6	7	8	9	10
Giá trị	3	5	8	7T	7	8T	4	9T	2*	3
Độ dài tabu	0	0	0	3	0	1	0	2	0	0

Vì thao tác 9 và thao tác 10, vốn có giá trị đánh giá tốt hơn, đều bị tabu, ta chỉ có thể chọn thao tác 8. Nhờ vậy thuật toán tránh nhảy về vùng tối ưu cục bộ cũ.

Như vậy ta thu được một nghiệm của bài toán: 0100100100.

Từ ví dụ này có thể tổng kết các bước của thuật toán tìm kiếm tabu.

Thuật toán 2.4.2: tìm kiếm tabu.

STEP1. Chọn một nghiệm ban đầu x_{now} và đặt bảng tabu $H := \emptyset$.

STEP2. Nếu thỏa quy tắc dừng, dừng tính toán. Ngược lại, trong lân cận $N(H, x_{\text{now}})$, chọn tập ứng viên $\text{Can_N}(x_{\text{now}})$ thỏa yêu cầu tabu. Trong $\text{Can_N}(x_{\text{now}})$, chọn nghiệm có giá trị đánh giá tốt nhất x_{next} , rồi đặt $x_{\text{now}} := x_{\text{next}}$. Cập nhật lịch sử H , lặp lại STEP2.

Một số vấn đề kỹ thuật. Các kỹ thuật trong tìm kiếm tabu bao gồm: đối tượng tabu, cấu tạo tập ứng viên, xác định độ dài tabu, xây dựng hàm đánh giá, quy tắc ân xá, quy tắc kết thúc, v.v. Chúng ảnh hưởng khá lớn tới hiệu quả thuật toán. Tất cả đều cần được phân tích theo từng bài toán cụ thể để thiết kế thuật toán hiệu quả hơn.

Đối tượng tabu là phần tử biến đổi bị cấm trong bảng tabu. Nó có thể ghi lại thay đổi đơn giản của nghiệm, ví dụ $(1101) \rightarrow (1011)$; cũng có thể ghi lại thay đổi của một thành phần nào đó của nghiệm, như bài xâu 01 trong ví dụ 2.4; hoặc ghi lại thay đổi của giá trị mục tiêu $f(x)$.

Độ dài tabu là số lần lặp mà đối tượng bị cấm không được chọn. Độ dài càng lớn thì càng dễ nhảy khỏi vùng tối ưu cục bộ, nhưng cũng có thể làm tốc độ hội tụ của giá trị mục tiêu quá chậm và ảnh hưởng hiệu quả thuật toán. Với bài “Xâu 01”, tôi dùng độ dài tabu không cố định: khi giá trị mục tiêu tốt nhất hiện tại càng nhỏ, độ dài tabu tương ứng tăng lên, có lợi cho việc nhảy khỏi nghiệm tối ưu cục bộ. Ngược lại, ở đầu quá trình chạy thuật toán, độ dài tabu nhỏ hơn để giá trị mục tiêu hội tụ nhanh, khiến thuật toán ở giai đoạn này gần với thuật toán tham lam hơn.

Hàm đánh giá là công thức dùng để đánh giá và chọn phần tử trong tập ứng viên. Trong bài “Xâu 01”, ta dùng chính giá trị mục tiêu $f(x)$ làm hàm đánh giá, điều này tương đối dễ hiểu. Nhưng trong một số bài toán, độ phức tạp tính giá trị mục tiêu quá lớn; nếu dùng nó làm hàm đánh giá thì thuật toán chạy chậm. Khi đó cần thiết kế một hàm đánh giá thay thế. Hàm đánh giá thay thế không chỉ phải có độ phức tạp tính toán thấp, mà còn phải phản ánh được một số đặc trưng của hàm mục tiêu.

Quy tắc ân xá xử lý các tình huống trong quá trình lặp khi toàn bộ đối tượng trong tập ứng viên đều bị tabu, hoặc có một đối tượng bị tabu nhưng nếu giải cấm thì giá trị mục tiêu sẽ giảm rất mạnh. Trong những trường hợp như vậy, để đạt tối ưu toàn cục, ta cho phép một số đối tượng tabu được chọn trở lại. Phương pháp này gọi là ân xá. Trong bài “Xâu 01”, nếu giá trị đánh giá của một thao tác bị tabu thấp hơn giá trị mục tiêu tốt nhất đã từng xuất hiện, $f_{\min}$, thì có thể giải cấm nó. Quy tắc ân xá này có thể nhanh chóng đưa nghiệm hiện tại tiến gần nghiệm cuối cùng khi $f(x)$ nhỏ.

Bước 6, $x_5 = (1100101001), f(x_5) = 2$	1	2	3	4	5	6	7	8	9	10
Giá trị	1*	3	6	5T	5	6	5	5T	4T	4
Độ dài tabu	0	0	0	2	0	0	0	1	3	0

Bước 7, $x_6 = (0100101001), f(x_6) = 1$	1	2	3	4	5	6	7	8	9	10
Giá trị	2T	5	4	4T	4	5	4	4	3T	3*
Độ dài tabu	3	0	0	1	0	0	0	0	2	0

Bước 8, $x_7 = (0100101000), f(x_7) = 3$	1	2	3	4	5	6	7	8	9	10
Giá trị	4T	7	6	6	6	7	6	3*	2T	1T
Độ dài tabu	2	0	0	0	0	0	0	0	1	3

Phạm vi áp dụng của tìm kiếm tabu. Chương trình của thuật toán tìm kiếm tabu tương đối đơn giản. Trong thi đấu, có thể dùng giới hạn thời gian của đề làm quy tắc kết thúc, qua đó tận dụng tối đa thời gian được cho và bù đắp nhược điểm của thuật toán tham lam. Nó thích hợp để giải một số bài toán tối ưu tổ hợp chuẩn, như bài gán nhãn bản đồ, bài quy hoạch khu xả lũ trong tài liệu [7] của Zhou Tianling. Với các bài toán khả thi giống bài “Xâu 01”, yêu cầu là bài phải tương đối dễ chuyển thành bài toán tối ưu, đồng thời phải có nhiều nghiệm khả thi.

2.2 Phương pháp ngẫu nhiên hóa

Ngẫu nhiên hóa không phải là một thuật toán hoàn chỉnh. Trong quá trình giải bài toán phi tối ưu, nó cần kết hợp với tham lam, tìm kiếm cục bộ và các thuật toán khác. Tác dụng của nó thể hiện ở hai mặt: với bài toán khả thi, nâng cao hiệu quả và tính ổn định của thuật toán; với bài toán tối ưu, làm kết quả tính toán của thuật toán gần nghiệm tối ưu hơn.

2.2.1 Ứng dụng ngẫu nhiên hóa trong tham lam

Vì độ phức tạp thời gian của thuật toán tham lam rất thấp, ứng dụng ngẫu nhiên hóa chủ yếu là đưa nhân tố ngẫu nhiên vào việc chọn chiến lược trong thuật toán, rồi thông qua việc chạy thuật toán lặp lại nhiều lần để thu được một nghiệm tốt nhất. Điều này thể hiện rất rõ và đem lại hiệu quả tốt trong thuật toán ngẫu nhiên hóa cho các bài “Tính toán song song” và “Cải tạo mê cung”, nên ở đây không trình bày thêm.

2.2.2 Ứng dụng ngẫu nhiên hóa trong tìm kiếm cục bộ

Trong tìm kiếm cục bộ, ngẫu nhiên hóa có hai cách dùng: một là khi chọn nghiệm khả thi ban đầu, hai là khi trong quá trình tìm kiếm có nhiều chiến lược cùng trọng số.

Nếu dùng phương pháp ngẫu nhiên để sinh nghiệm khả thi ban đầu, ta có thể tránh hiệu quả các bẫy tối ưu cục bộ được tạo sẵn. Điều này có thể nâng cao hiệu quả trung bình của tìm kiếm cục bộ.

Nếu trong quá trình tìm kiếm cục bộ xuất hiện một số chiến lược có cùng giá trị đánh giá, ta cũng có thể dùng phương pháp ngẫu nhiên để chọn một trong số đó, qua đó cải thiện hiệu quả thuật toán.

Hiệu quả của hai kỹ thuật trên trong bài “Xâu 01” được trình bày ở bảng 2.

Dấu * nghĩa là trong 3.85% thể hiện, thuật toán rơi vào bẫy tối ưu cục bộ và không thoát ra được.

3 Tổng kết

Thuật toán phi tối ưu không chỉ có thể áp dụng để giải bài toán phi tối ưu mà còn có thể áp dụng cho các bài toán tối ưu khác. Nó có các ưu điểm sau:

Bước 9, $x_8 = (0100101100), f(x_8) = 3$	1	2	3	4	5	6	7	8	9	10
Giá trị	4T	7	6	6	6	8	0*	3T	6	4T
Độ dài tabu	1	0	0	0	0	0	0	3	0	2

Số lần lặp tìm kiếm	Sequence.003	Sequence.004	Sequence.005
Tìm kiếm tabu thông thường	1092	1357	758
Chọn ngẫu nhiên giữa thao tác cùng giá trị	854 (3.85%)*	726 (0.00%)	954 (0.00%)
Mức tăng hiệu quả	21.8%	46.5%	-25.9%
Nghiem ban đầu sinh ngẫu nhiên	27	13	19
Mức tăng hiệu quả	97.5%	99.0%	97.5%

Table 2: Ảnh hưởng của ngẫu nhiên hóa trong bài “Xâu 01”

1. Phạm vi áp dụng rộng. Thuật toán phi tối ưu là một loại thuật toán phổ dụng, không có yêu cầu đặc biệt đối với hàm mục tiêu hay các tính chất khác của đề bài.
2. Khi dùng thuật toán phi tối ưu để giải bài toán tối ưu, ta không cần kỹ thuật quá mạnh hoặc hiểu bài toán quá sâu. Điều này đặc biệt đúng với một số bài tối ưu hóa tìm kiếm vốn cần kỹ thuật rất đặc thù. Trong thuật toán phi tối ưu, quá trình tính toán và độ phức tạp chương trình đều tương đối đơn giản, đồng thời có thể nhanh chóng thu được một nghiệm thỏa đáng.

Việc áp dụng rộng rãi các bài yêu cầu nghiệm không tối ưu và các thuật toán phi tối ưu trong thi đấu vẫn chỉ là chuyện của vài năm gần đây. Nhiều lý thuyết còn chưa chín muồi và còn thiếu kinh nghiệm thực tiễn cần thiết, vì vậy tổng kết trong bài viết này còn rất nông. Những thuật toán được giới thiệu ở đây chỉ là vài thuật toán trong số nhiều thuật toán phi tối ưu; ngoài ra còn có các phương pháp tính toán tối ưu hóa hiện đại mới nổi như mô phỏng luyện kim, thuật toán di truyền, mạng nơ-ron nhân tạo, v.v. Việc áp dụng các thuật toán này trong thi đấu vẫn cần mọi người tiếp tục tìm tòi, thực hành và tổng kết.

Tài liệu tham khảo

1. *Olympic Tin học* (1998–1999). Hội Máy tính Trung Quốc.
2. *Tuyển tập luận văn xuất sắc của đội tuyển huấn luyện IOI 1998 Trung Quốc*.
3. *Tuyển tập luận văn đội tuyển huấn luyện IOI 1999 Trung Quốc*.
4. Liu Fusheng, Wang Jiande. *Hướng dẫn Olympic Tin học Quốc tế dành cho thanh thiếu niên: tìm kiếm trí tuệ nhân tạo và thiết kế chương trình*. Nhà xuất bản Công nghiệp Điện tử, 1993.
5. Xing Wenxun, Xie Jinxing. *Các phương pháp tính toán tối ưu hóa hiện đại*. Nhà xuất bản Đại học Thanh Hoa, 1999.
6. Wang Shuhe. *Cơ sở mô hình toán học*. Nhà xuất bản Đại học Khoa học và Công nghệ Trung Quốc.
7. *Bài tập lần thứ ba của đội tuyển huấn luyện IOI 1999 Trung Quốc*.

A NOI 1999: Xâu 01

Phụ lục dưới đây là chương trình Pascal trong bản gốc. Các chú thích tiếng Trung trong chương trình đã được dịch sang tiếng Việt không dấu để tiện biên dịch trong môi trường mã nguồn cũ.

```
{ $A+, B-, D+, E+, F-, G-, I+, L+, N-, O-, P-, Q-, R-, S+, T-, V+, X+ }
{ $M 16384, 0, 655360 }
program noi99_1;
const
  ifilename:string='sequence.005';
```

```

type
  polytype=array[1..1000] of 0..1;
  pathtype=array[0..1,1..1000] of integer;
var
  timer:longint;
  len:integer;           { do dai bang tabu }
  n,n0,n1:integer;
  a0,b0,l0,a1,b1,l1:integer;
  poly:polytype;        { day ky tu hien tai }
  path:pathtype;         { so ky tu 0 va 1 lien tiep trong poly }
  now:longint;           { gia tri muc tieu hien tai }
  total:integer;         { so lan lap }

procedure init;
var
  f:text;
  i,j:integer;
begin
  assign(f,ifilename); reset(f);
  readln(f,n,a0,b0,l0,a1,b1,l1);
  for i:=1 to n do       { sinh ngau nhien nghiem ban dau }
    poly[i]:=random(2);
  fillchar(path,sizeof(path),0);
  n1:=n-l1+1; n0:=n-l0+1;
  { tinh gia tri muc tieu cua nghiem ban dau }
  now:=0;
  for i:=1 to n1 do
    begin
      for j:=i to i+l1-1 do
        if poly[j]=1 then inc(path[1,i]);
        if path[1,i]<a1 then inc(now,a1-path[1,i]);
        if path[1,i]>b1 then inc(now,path[1,i]-b1);
      end;
    for i:=1 to n0 do
      begin
        for j:=i to i+l0-1 do
          if poly[j]=0 then inc(path[0,i]);
          if path[0,i]<a0 then inc(now,a0-path[0,i]);
          if path[0,i]>b0 then inc(now,path[0,i]-b0);
        end;
      close(f);
    end;

procedure changevalue(p:integer);      { sau khi thao tac vi tri p, sua path }
var
  i,t1,t2:integer;
begin
  if p>l1 then t1:=p-l1+1 else t1:=1;

```

```

if p<n1 then t2:=p else t2:=n1;
if poly[p]=0 then begin
  for i:=t1 to t2 do dec(path[1,i]);
end else begin
  for i:=t1 to t2 do inc(path[1,i]);
end;

if p>l0 then t1:=p-l0+1 else t1:=1;
if p<n0 then t2:=p else t2:=n0;
if poly[p]=1 then begin
  for i:=t1 to t2 do dec(path[0,i]);
end else begin
  for i:=t1 to t2 do inc(path[0,i]);
end;
end;

function calcvalue(p:integer):longint; { gia tri danh gia sau khi lat vi tri p }
var
  t:longint;
  i,t1,t2:integer;
begin
  t:=now;
  if p>l1 then t1:=p-l1+1 else t1:=1;
  if p<n1 then t2:=p else t2:=n1;
  if poly[p]=1 then
    for i:=t1 to t2 do
      begin
        if path[1,i]>b1 then dec(t);
        if path[1,i]<=a1 then inc(t);
      end
  else
    for i:=t1 to t2 do
      begin
        if path[1,i]>=b1 then inc(t);
        if path[1,i]<a1 then dec(t);
      end;

  if p>l0 then t1:=p-l0+1 else t1:=1;
  if p<n0 then t2:=p else t2:=n0;
  if poly[p]=0 then
    for i:=t1 to t2 do
      begin
        if path[0,i]>b0 then dec(t);
        if path[0,i]<=a0 then inc(t);
      end
  else
    for i:=t1 to t2 do
      begin

```

```

        if path[0,i]>=b0 then inc(t);
        if path[0,i]<a0 then dec(t);
    end;
    calcvalue:=t;
end;

procedure compute;                                { qua trinh tim kiem tabu }
var
    i,mi,link:integer;
    sign:array[0..1000] of boolean;                { vi tri co nam trong bang tabu khong }
    list:array[1..200] of integer;                  { bang tabu }
    min,t,mmin:longint;                             { min: gia tri tot nhat trong ung vien;
                                                    mmin: muc tieu tot nhat tung dat }
    ll:array[0..5] of integer;                       { do dai tabu theo tung giai doan mmin }
begin
    fillchar(sign,sizeof(sign),false);
    fillchar(list,sizeof(list),0);
    link:=0; mmin:=maxlongint;
    len:=trunc(n/10);
    ll[0]:=trunc(n/5); ll[1]:=trunc(n/6); ll[2]:=trunc(n/7);
    ll[3]:=trunc(n/8); ll[4]:=trunc(n/9);
    total:=0;
    while now>0 do
        begin
            inc(total);
            if total>2000 then exit;
            if now<mmin then mmin:=now;
            if mmin<1000 then len:=ll[mmin div 200];
            min:=maxlongint;
            { chon thao tac toi uu trong tap ung vien }
            for i:=1 to n do
                if not sign[i] then
                    begin
                        t:=calcvalue(i);
                        if (t<min) {or ((t=min) and (random(2)=0))} then
                            begin
                                min:=t; mi:=i;
                            end;
                        end;
                    else begin
                        t:=calcvalue(i);
                        if (t<mmin) and (t<min) then begin          { quy tac an xa }
                            min:=t; mi:=i;
                        end;
                    end;
            now:=min;
            poly[mi]:=1-poly[mi];
            changevalue(mi);

```

```

        inc(link);
        if sign[mi] then continue;           { truong hop an xa }
        { sua bang tabu }
        sign[list[link]]:=false;
        list[link]:=mi; sign[mi]:=true;
        if link=len then link:=0;
    end;
    for i:=1 to n do write(poly[i]);
    writeln;
end;

begin
    randomize;
    timer:=meml[$40:$6c];
    init;
    compute;
    writeln('Time=',(meml[$40:$6c]-timer)/18.2:0:2);
end.

```